

Term Project

화재경보기



2021707085 / 박찬희

2023707002 / 윤혜린(조장)

2023707028 / 강서희

2023707059 / 김가현

목 차

1. 프로젝트 제안 배경 및 목표
2. 프로젝트 관련 이론 및 사용 부품
3. 작품 제작 과정
 - 시행착오 및 해결방안
 - 회로도 및 코드
4. 최종결과물
5. 고찰
 - 각 시행착오 단계에서의 고찰
6. 역할 분담 및 프로젝트 진행과정

1. 프로젝트 제안 배경 및 목표

- 프로젝트 제안 배경

현대 사회에서 화재는 가정, 산업, 공공장소 등 다양한 환경에서 심각한 인명 및 재산 피해를 초래해 화재 발생 초기의 신속한 탐지와 경고를 통해 피해를 최소화하는 것이 중요하다.

이런 점에서 화재경보기는 온도 상승, 연기 감지 등 특정 조건을 실시간으로 모니터링하여 위험을 조기에 알리고, 적절한 대응을 가능하게 한다.

그래서 이번 프로젝트는 온도 센서와 SPI 통신을 활용하여 데이터를 표시하고, LED와 부저로 화재 경고를 제공하는 간단하고 실용적인 화재경보기를 설계함으로써 화재 위험을 줄이고 보다 안전한 환경을 조성하는 데 기여하고자 한다.

- 목표

본 프로젝트에서는 SPI(Serial Peripheral Interface) 통신을 사용하여 화재 경보기에 필요한 데이터를 표시하고 경보 장치를 작동하는 방법을 탐구한다. 주요 목표는 기본 통신 프로토콜이 실제 응용 분야에서 어떻게 작동하는지 이해하고 이를 통해 실시한 데이터 전송 및 제어 기능을 구현하는 것이다.

SPI 통신을 구현함으로써 온도 센서에서 7-segment 디스플레이, LED, 부저로 데이터를 전송할 수 있으며, 이 설정은 화재 경보 시스템에서 SPI의 실제 적용을 보여준다. 또한 온도가 특정 임계값을 초과할 경우 LED와 부저를 사용하여 경고를 제공함으로써, 사용자에게 시각적, 청각적 알림을 실시간으로 전달 가능하다. 또한 온도센서에서 디스플레이로 직접 4-bit 입력 신호를 전송할 수 있어 온도 판독값을 실시간으로 시각화할 수 있다. 이 설정은 온도 측정 장치에서 SPI의 실제 적용을 보여줄 뿐만 아니라 특정 온도 범위 내에서 정확한 최신 정보 표시에 필요한 실시간 데이터 전송을 강조한다.

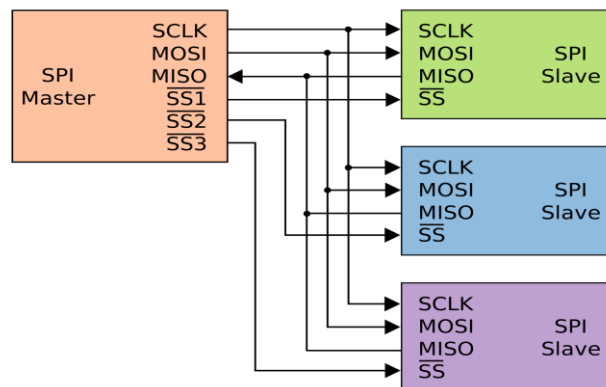
이번 프로젝트는 기초전자회로실험 2에서 배운 개념을 적용하고 검증하고자 선택하게 되었다. 프로그래밍과 회로 설계를 결합한 다목적 플랫폼인 Arduino를 통합함으로써 통제된 환경에서 여러 개념을 탐구하고 안전한 저전압 실험을 실행할 수 있다. 이를 통해 SPI 프로토콜, 데이터 표시 방법 및 경고 메커니즘의 실습 경험을 통해 화재 경보기 시스템의 통신 원리와 실제 작동에 대한 이해를 향상시킬 수 있다.

궁극적으로 이 프로젝트를 통해 수업 시간에 공부한 전자원리의 실용적인 유용성을 강조하고 일상 생활에서 안전하고 유용한 솔루션을 개발하여 이론적 지식과 실제 적용을 연결하려 한다.

2. 프로젝트 관련 이론 및 사용 부품

- 프로젝트 관련 이론

아래의 다이어그램은 SPI(Serial Peripheral Interface) 통신 설정의 전체적인 기본 구조를 보여주며, 마스터 장치가 여러 Slave 장치와 연결되는 방식을 보여준다.



SPI protocol에서는 단일 마스터와 하나 이상의 Slave 장치 간에 통신이 발생한다. 왼쪽에 SPI 마스터, 오른쪽에 3개의 SPI Slave를 나타내고, 각 Slave에는 마스터의 전용 SS(Select) 라인이 있다.

1. [SCLK]

마스터 장치는 마스터와 Slave 간의 데이터 전송을 동기화하는 SCLK 신호를 생성한다. 이 라인은 연결된 모든 장치 간에 공유되어 데이터 전송이 마스터의 clock 주기와 동기화 되어 발생하도록 보장한다.

2. [SS]

각 Slave 장치에는 마스터에 연결되는 SS 라인이 있다. 특정 Slave와 통신하기 위해 마스터는 해당 SS 라인을 낮은 상태로 설정하여 데이터 교환을 위해 해당 Slave를 활성화한다. 이미지에서 마스터에는 3개의 Slave를 독립적으로 제어하기 위한 3개의 SS 라인(SS1, SS2, SS3)이 있다.

3. [MOSI]

이 라인은 마스터에서 선택한 Slave로 데이터를 전달한다. 모든 Slave 장치는 이 라인을 공유하지만 SS 라인에 의해 선택된 Slave만 MOSI의 데이터를 읽고 그렇지 않으면 무시한다.

4. [MISO]

MISO 라인을 사용하면 Slave가 데이터를 마스터로 다시 보낼 수 있다. MOSI와 마찬가지로 이 회선은 모든 Slave 간에 공유되지만 회선 충돌을 피하기 위해 선택된 Slave만 데이터를 보낸다.

SPI 설정과 관련하여 D-Flip-Flop은 데이터를 안정화하기 위해 각 Slave 또는 시스템 내에서 사용된다. 수신 또는 전송 온도 판독값이 캡처되면 D-Flip-Flop은 다음 Clock 사이클까지 해당 값을 안정적으로 유지하여 7-segment 디스플레이에 안정적인 출력 표시를 제공한다. 또한 S(설정) 및 R(재설정) 신호를 통해 온도 표시를 즉시 재설정하거나 설정할 수 있어 판독값 변동으로 인한 표시 오류를 방지하는데 도움이 된다.

- 사용 부품

Arduino uno board

Ic74165

Ic74595

CD4511

1k 저항

DS18B20

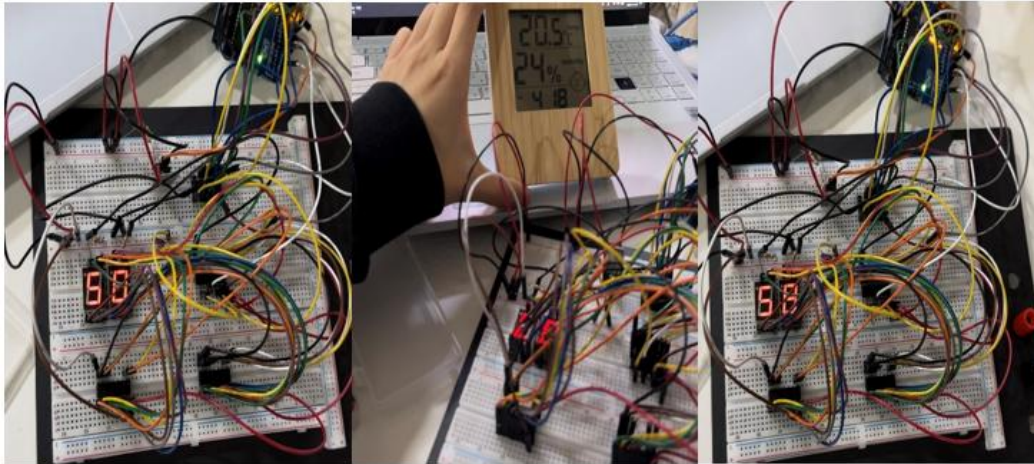
7-segment

Red LED

피에조 부저

3. 작품 제작 과정

- 시행착오1 및 해결과정

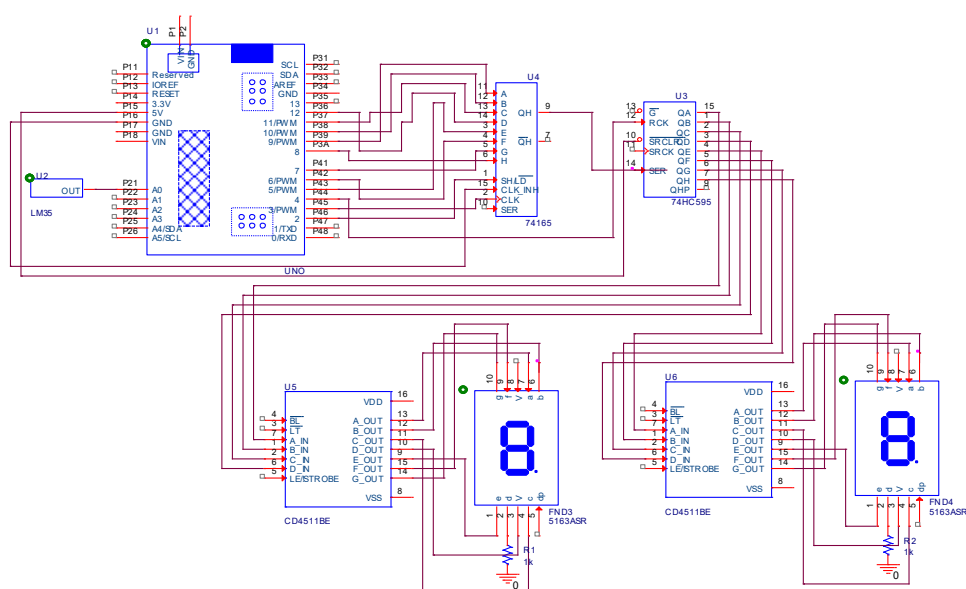


문제점: 현재 설정에서는 Arduino가 온도를 성공적으로 읽고 출력하지만 7-segment 디스플레이에는 정확한 값이 표시되지 않는다. 또 7-segment 디스플레이의 출력이 일관되지 않는다.

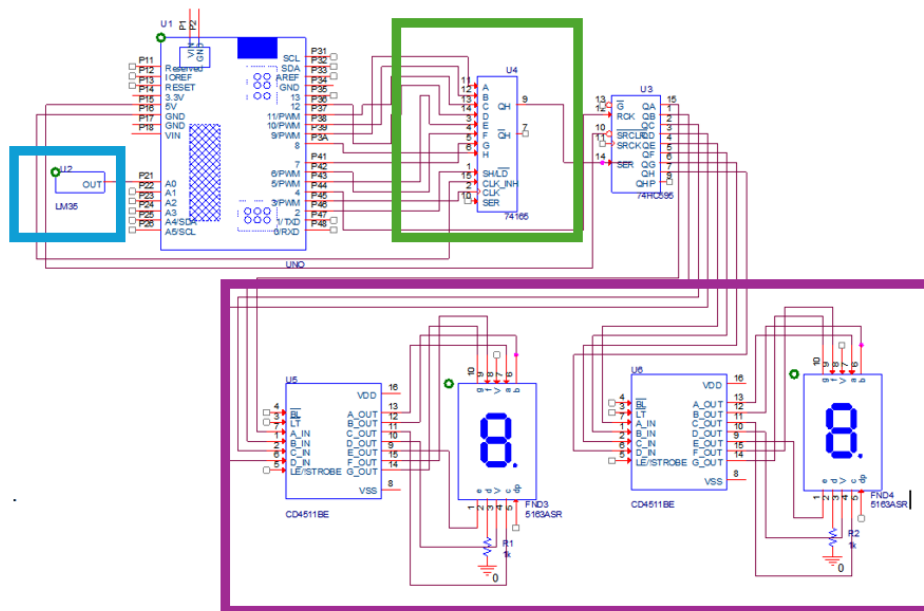
해결하고자 하는 것: 현재의 경우 세그먼트에 임의의 숫자가 나타나거나 Arduino의 온도 변화 데이터가 디스플레이에 올바르게 반영되지 않기에 7-segment 디스플레이에서 정확하고 안정적인 온도 판독을 보장하고자 한다.

기존 회로도와 변경된 회로도1

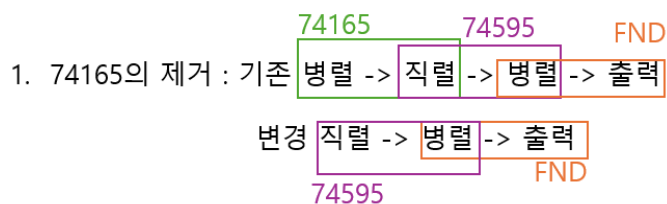
-기존 회로도



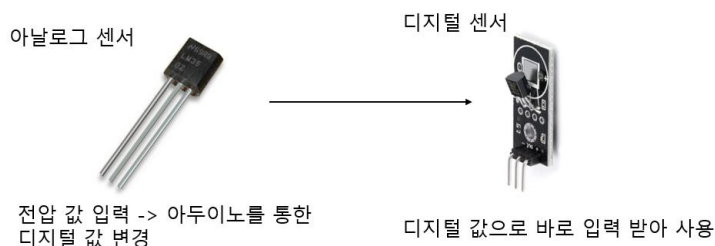
회로 변경점 정리



(1) 74165 IC 제거(초록색 부분): 초기 설계에서는 74165 IC를 사용하여 병렬 입력 데이터를 74595를 통해 직렬로 출력할 계획이었으나, 코드를 활용하여 Arduino 핀 하나를 통해 직렬 입력이 가능하다는 점을 확인하였다. 이에 따라 74595의 데이터 핀을 Arduino의 11번 핀에 연결하여 직렬 데이터 입력을 처리하도록 변경하였다.

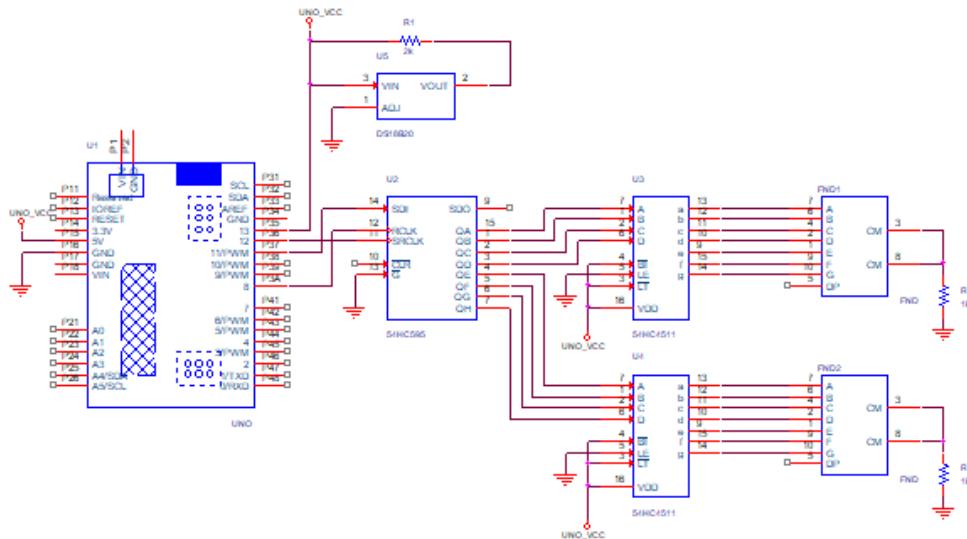


(2) 온도 센서 변경(파란색 부분): 초기 계획에서는 LM35 온도 센서를 통해 아날로그 전압값을 읽어 Arduino에서 디지털 데이터로 변환하여 사용할 예정이었으나, DS18B20 센서를 사용하여 디지털 데이터를 바로 수신하는 방식으로 변경하였다. 이를 통해 데이터 변환 과정이 간소화되었으며, 구현 효율성이 향상되었다.



(3) 4511 IC 연결 오류 수정(보라색 부분): 4511 IC의 데이터 시트를 확인한 결과, 3번 및 4번 핀을 VCC에, 5번 핀을 GND에 연결하지 않으면 정상적으로 동작하지 않는다는 점을 확인하였다. 이를 반영하여 3번 및 4번 핀은 VCC에, 5번 핀은 GND에 연결하였으며, 기타 자잘한 연결 오류도 모두 수정하였다.

-변경 회로도1



기존 회로에서 변경되어야 할 것을 정리하여 다음과 같이 회로도를 전면 수정하였다.

기존 아두이노 코드와 변경된 아두이노 코드1

기존 아두이노 코드

이 Arduino 코드는 온도 센서로부터 값을 읽어 7-segment 디스플레이에 표시하는 시스템을 구현한다. 온도 판독과 통신은 OneWire와 DallasTemperature 라이브러리를 통해 이루어지며, IC 74595 시프트 레지스터를 사용하여 디스플레이의 세그먼트를 제어한다.

코드 실행 초기에는 OneWire 객체를 생성해 온도 센서와의 통신 프로토콜을 설정하고, DallasTemperature 객체를 초기화하여 온도 데이터를 처리할 준비를 한다. 설정 단계에서는 직렬 통신을 활성화하고, 온도 센서를 초기화하며, 데이터를 디스플레이로 전송하기 위한 출력 핀들을 구성한다.

온도 값은 displayTemperature 함수에서 처리된다. 함수는 온도 센서로부터 데이터를 읽어 값을 10의 자리와 1의 자리로 나누고, 이를 BCD 형식으로 변환한 후 shiftOutData 함수를 통해 시프트 레지스터에 전송한다. 이후 Latch 핀을 HIGH로 설정하여 변환된 값이 7-segment 디스플레이에 표시되도록 한다.

기초전자실험 2 최종보고서

```
#include <DallasTemperature.h>
#include <OneWire.h>

#define ONE_WIRE_BUS 2
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

#define LATCH_PIN 10 // RCK (74HC595 Pin 12)
#define CLOCK_PIN 11 // SRCLK (74HC595 Pin 11)
#define DATA_PIN 12 // SER (74HC595 Pin 14)

// Function to send data to the shift register
void shiftOutData(uint8_t data) {
    digitalWrite(LATCH_PIN, LOW);
    shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, data);
    digitalWrite(LATCH_PIN, HIGH);
}

// BCD values for 0-9 to send to CD4511BE
byte bcdDigits[] = {
    0b00000000, // 0
    0b00000001, // 1
    0b00000010, // 2
    0b00000011, // 3
    0b00000100, // 4
    0b00000101, // 5
    0b00000110, // 6
    0b00000111, // 7
    0b00001000, // 8
    0b00001001 // 9
};

void setup() {
    Serial.begin(9600);
    sensors.begin();

    pinMode(LATCH_PIN, OUTPUT);
    pinMode(CLOCK_PIN, OUTPUT);
    pinMode(DATA_PIN, OUTPUT);
}

void displayTemperature(int temperature) {
    int tens = temperature / 10; // Extract tens digit
    int units = temperature % 10; // Extract units digit

    // Send tens digit to the first 7-segment display
    shiftOutData(bcdDigits[tens]);
    delay(5);

    // Send units digit to the second 7-segment display
    shiftOutData(bcdDigits[units]);
    delay(5);
}

void loop() {
    sensors.requestTemperatures();
    int tempC = (int)sensors.getTempCByIndex(0);

    Serial.print("Temperature: ");
    Serial.print(tempC);
    Serial.println(" °C");

    displayTemperature(tempC); // Display temperature on 7-segment displays

    delay(1000);
}
```

변경된 아두이노 코드1

```
#include <DallasTemperature.h>
#include <OneWire.h>

#define ONE_WIRE_BUS 13 // DS18B20 데이터 핀 연결

// 74HC595 핀 정의
const int DATA_PIN = 11;
const int CLOCK_PIN = 12;
const int LATCH_PIN = 8;

// DS18B20 센서 초기화
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

// BCD 코드 테이블 (0~9)
const byte BCD_CODES[10] = {
  0b00000000, // 0
  0b00000001, // 1
  0b00000010, // 2
  0b00000011, // 3
  0b00000100, // 4
  0b00000101, // 5
  0b00000110, // 6
  0b00000111, // 7
  0b00001000, // 8
  0b00001001 // 9
};

// fndData를 시프트 레지스터로 전송하는 함수
void sendToFND(int num1, int num2) {
  byte fndData = 0x00;

  // 첫 번째 숫자 (num1) 출력 (1번 FND)
  fndData |= (BCD_CODES[num1] & 0x0F);

  // 두 번째 숫자 (num2) 출력 (2번 FND)
  fndData |= (BCD_CODES[num2] << 4);
  // 시프트 레지스터에 데이터 전송
  digitalWrite(LATCH_PIN, LOW);
  shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, fndData);
  digitalWrite(LATCH_PIN, HIGH);
}

void setup() {
  Serial.begin(9600); // 시리얼 통신 시작
  sensors.begin();    // 온도 센서 초기화
  pinMode(DATA_PIN, OUTPUT);
  pinMode(CLOCK_PIN, OUTPUT);
  pinMode(LATCH_PIN, OUTPUT);
}
```

```
void loop() {  
  sensors.requestTemperatures(); // 온도 읽기  
  
  // 온도 값 얻기  
  float temperatureC = sensors.getTempCByIndex(0); // 섭씨 온도 읽기  
  int tempCInt = (int)temperatureC; // 섭씨 온도 정수부분  
  
  // 시리얼 모니터로 온도 출력  
  Serial.print("Temperature in Celsius: ");  
  Serial.println(temperatureC); // 현재 센서에서 받은 섭씨 온도 출력  
  
  // 온도 값을 FND에 출력 (2개의 FND에 각 자리 수 표시)  
  sendToFND(tempCInt / 10, tempCInt % 10); // 섭씨 온도 2자리 수  
  delay(1000); // 1초 간격으로 업데이트  
}
```

이 코드는 DS18B20 온도 센서를 사용하여 섭씨 온도를 측정하고 7-segment 디스플레이에 표시하도록 설계했다. 온도 센서와의 통신은 OneWire와 DallasTemperature 라이브러리를 사용하며, 센서 데이터 핀은 Arduino의 13번 핀에 연결한다. 디스플레이 제어는 74HC595 시프트 레지스터를 활용하며, 데이터 핀(12->11), 클록 핀(11->12), 래치 핀(10->8)을 통해 동작한다.

온도 측정 과정에서 센서로부터 읽어온 데이터는 실수형에서 정수형으로 변환한 후 10의 자리와 1의 자리로 나누어 BCD 코드로 변환한다. 변환된 데이터는 시프트 레지스터를 통해 디스플레이로 전송되며, 래치 핀을 사용하여 출력을 갱신한다. 또한, 측정된 온도 값은 시리얼 모니터를 통해 출력하여 디버깅 및 모니터링이 가능하도록 설계되었다. 또한, 측정된 온도 값은 시리얼 모니터에 출력하여 디버깅 및 모니터링이 가능하도록 설계되었다.

이 코드는 1초 간격으로 온도를 갱신하며, 두 자리 데이터를 한 번에 처리하도록 최적화된 sendToFND 함수를 사용해 FND 디스플레이에 실시간으로 반영한다. 핀 연결과 데이터 전송 방식이 간소화되었고 코드 구조가 개선되어 유지보수와 확장성이 향상되었다.

해결 후 시연 결과



시연 결과

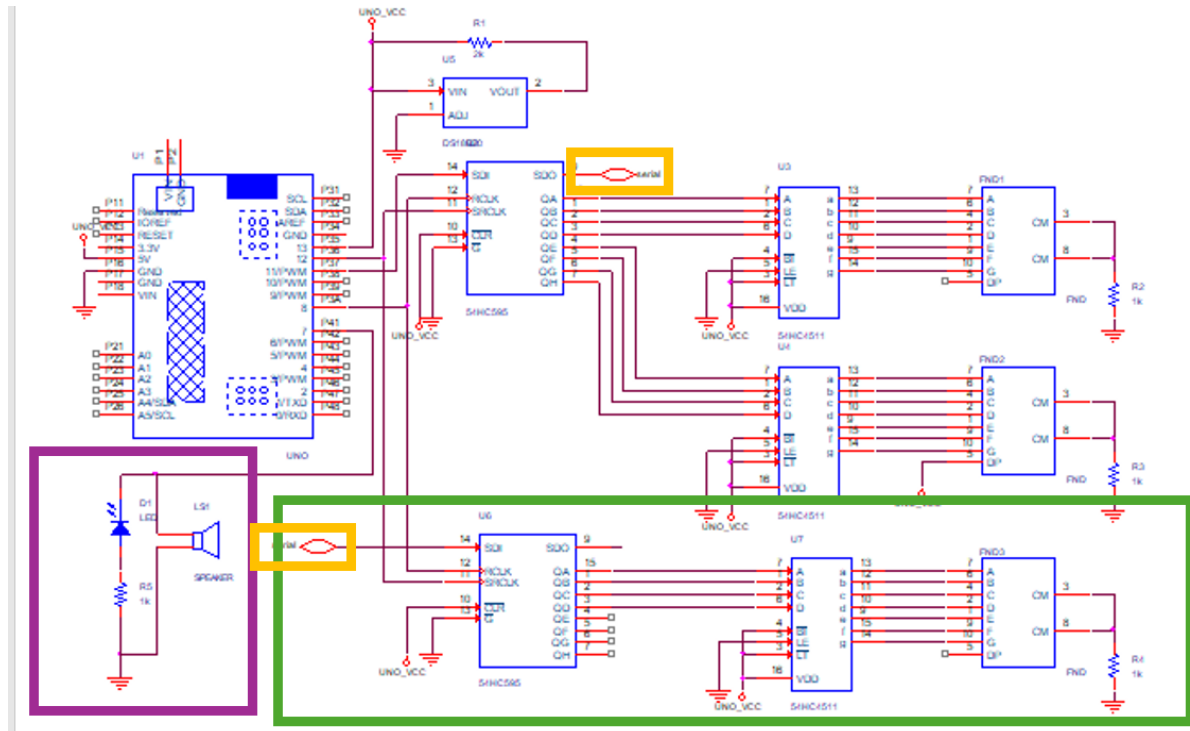
Arduino가 온도를 성공적으로 읽고 7-segment 디스플레이에는 정확한 값이 출력함을 볼 수 있다. 또 7-segment 디스플레이의 출력이 일관되어 안정적인 온도판독을 진행함을 알 수 있다.

- 시행착오2 및 해결과정

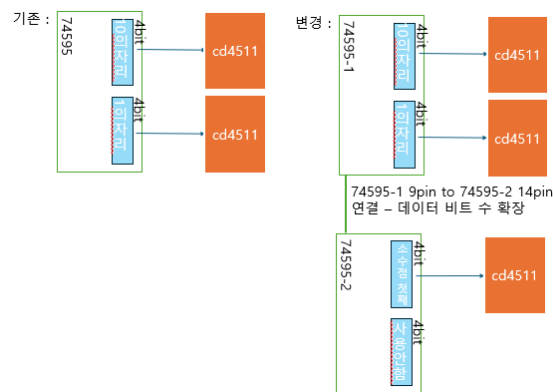
문제점: 현재 설정에서는 Arduino가 온도를 성공적으로 읽고 7-segment 디스플레이에까지 출력 가능하다. 하지만 아두이노 시리얼모니터에서는 소수점자리까지 온도를 판독하는데 우리가 지금까지 제작한 회로도는 소수점자리를 표현하지 못한다. 또 화재경보기로서 기능을 수행하지 못한다.

해결하고자 하는 것: 아두이노 시리얼 모니터에서와 마찬가지로 7-segment 디스플레이에도 소수점자리까지의 온도를 정확하게 표현해 완성도를 높이고 부저와 LED를 사용하여 주변의 온도가 30°C 이상이 되면 LED가 깜빡이고 부저에 소리가 울리도록 하여 화재경보기로서 기능을 수행하도록 하는 것이 목표이다.

회로 변경점 정리 사진



(1) 74595 IC 시프트 레지스터 확장(노란색 부분): 기존 회로에서는 단일 74595 시프트 레지스터를 사용해 CD4511 디코더와 7-segment 디스플레이를 제어했지만, 변경된 회로에서는 74595 두 개를 직렬로 연결하여 데이터 비트를 확장하였다. 첫 번째 74595의 9번 핀(Q7S)을 두 번째 74595의 14번 핀(DATA IN)에 연결해 직렬 데이터 흐름을 구성함으로써, 기존 8비트에서 16비트로 데이터 전송이 확장되었다. 이를 통해 두 개의 CD4511 디코더를 각각 독립적으로 제어할 수 있어, 더 많은 7-segment 디스플레이를 연결하거나 제어 범위를 확대할 수 있게 되었다. 데이터 확장을 통해 기존 회로 대비 유연성과 확장성이 크게 향상되었다.

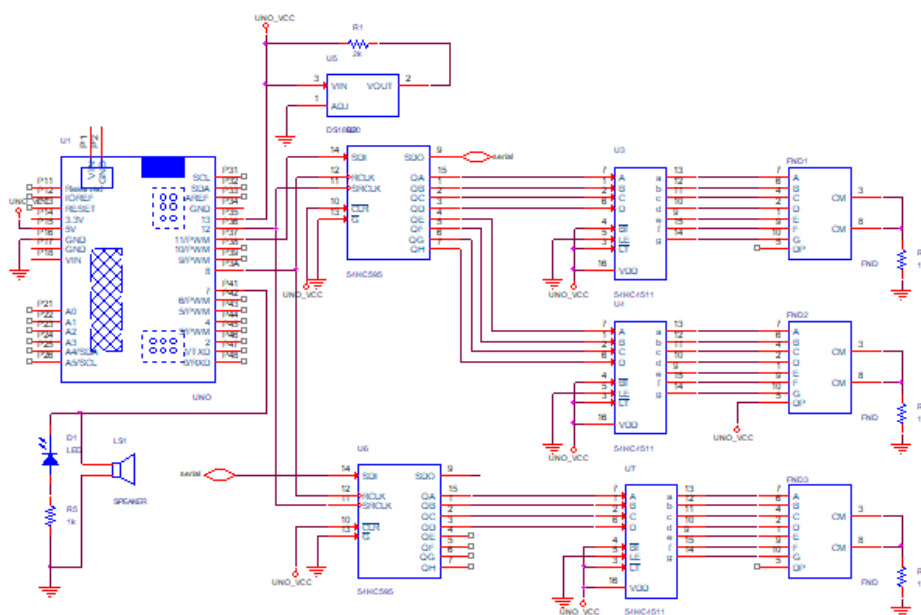


[74595 확장]

(2) 화재 경보를 위한 부저 및 LED 추가 (보라색 부분): 화재 경고 기능을 구현하기 위해 회로에 부저와 LED를 추가하였다. 부저는 경고음을 통해 화재 상황을 알리고, LED는 시각적인 경고 신호를 제공한다. 회로의 안정적인 작동을 위해 풀다운 저항 방식을 사용하여 입력 핀이 미연결 상태일 때 발생할 수 있는 노이즈를 방지하였고 이를 통해 부저와 LED의 신호가 명확히 동작하며, 화재 경고 시스템의 신뢰성을 높였다.

(3) 소수점 첫째 자리를 표시하기 위한 FND 추가 및 데이터 확장 (초록색 부분): 소수점 첫째 자리를 표시하기 위해 FND 한 개를 추가하고, 이를 제어하기 위해 CD4511 디코더와 데이터 전송 비트 수를 확장하는 74595 시프트 레지스터를 도입했다. 기존의 단일 74595로는 데이터 비트가 부족하여 추가된 FND를 제어하기 어렵기 때문에, 74595를 두 개 직렬 연결하여 데이터 전송 비트를 기존 8비트에서 16비트로 확장했다. 이를 통해 추가된 FND에 필요한 데이터를 안정적으로 전송할 수 있으며, 소수점 자리 표시가 가능해져 출력의 정확성과 표현 범위가 향상되었다.

-변경 회로도2



변경회로도1에서 변경되어야 할 것을 반영하여 다음과 같이 회로도를 전면 수정하였다.

변경된 아두이노 코드2

```
#include <DallasTemperature.h>
#include <OneWire.h>

#define ONE_WIRE_BUS 13 // DS18B20
#define BUZZER_PIN 7 // 부저 핀 정의

// 74HC595 핀 정의
const int DATA_PIN = 11;
const int CLOCK_PIN = 12;
const int LATCH_PIN = 8;

// DS18B20 센서 초기화
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

// BCD 코드 테이블 (0~9)
const byte BCD_CODES[10] = {
  0b00000000, // 0
  0b00000001, // 1
  0b00000010, // 2
  0b00000011, // 3
  0b00000100, // 4
  0b00000101, // 5
  0b00000110, // 6
  0b00000111, // 7
  0b00001000, // 8
  0b00001001 // 9
};

// FND에 데이터를 출력하는 함수
void sendToFND(int tens, int units, int decimal1) {
  byte fndData1 = 0x00; // (10의 자리, 1의 자리)
  byte fndData2 = 0x00; // (소수점 자리)

  // 첫 번째 시프트 레지스터 데이터 설정
  fndData1 |= (BCD_CODES[tens] & 0x0F); // 10의 자리
  fndData1 |= (BCD_CODES[units] << 4); // 1의 자리

  // 두 번째 시프트 레지스터 데이터 설정
  fndData2 = (BCD_CODES[decimal1] & 0x0F);
  // 소수점 자리
  // 래치 핀 LOW 설정
  digitalWrite(LATCH_PIN, LOW);

  // (소수점 자리)
  shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, fndData2);

  // (10의 자리, 1의 자리)
  shiftOut(DATA_PIN, CLOCK_PIN, MSBFIRST, fndData1);

  // 래치 핀 HIGH 설정
  digitalWrite(LATCH_PIN, HIGH);
}

void setup() {
  Serial.begin(9600); // 시리얼 통신 시작
  sensors.begin(); // 온도 센서 초기화

  // 핀 설정
  pinMode(DATA_PIN, OUTPUT);
  pinMode(CLOCK_PIN, OUTPUT);
  pinMode(LATCH_PIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  // 초기화 메시지
  Serial.println("온도 센서 및 FND 디스플레이 시작");
}

//코드백업
void loop() {
  sensors.requestTemperatures(); // 온도 읽기

  // 온도 값 얻기
  float temperatureC = sensors.getTempCByIndex(0);
  int tempCInt = (int)temperatureC;
  int tempDecimal1 = (int)((temperatureC - tempCInt) * 10);

  // 시리얼 모니터로 온도 출력
  Serial.print("Temperature in Celsius: ");
  Serial.println(temperatureC);

  // 온도 값을 FND에 출력
  sendToFND(tempCInt / 10, tempCInt % 10, tempDecimal1);

  // 30도 이상일 경우 부저 위웅위웅 소리 발생
  if (temperatureC >= 30) {
    for (int i = 0; i < 3; i++) { // 3번 반복
      digitalWrite(BUZZER_PIN, HIGH); // 부저 켜기
      delay(500); // 0.5초 대기
      digitalWrite(BUZZER_PIN, LOW); // 부저 끄기
      delay(500); // 0.5초 대기
    }
  }

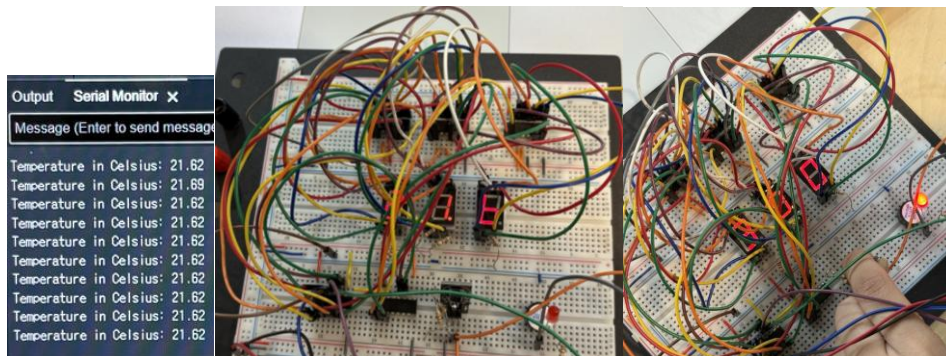
  delay(1000); // 1초 간격으로 업데이트
}
```

변경코드2는 DS18B20 온도 센서를 이용하여 측정된 온도를 소수점 첫째 자리까지 표시하고, 특정 온도 이상에서는 부저를 통해 경고음을 발생시키는 기능을 추가한 코드이다. 주요 변경 사항으로는 부저 핀(7번)을 추가하여 온도가 30°C 이상일 경우 0.5초 간격으로 3회 울리도록 구현한 점과, 기존 온도 표시 기능을 확장하여 소수점 첫째 자리까지 정확히 출력하는 기능을 포함한 점이 있다.

코드에서는 정수부와 소수점 첫째 자리를 분리해 각각 다른 FND에 출력하며, 이를 위해 sendToFND 함수가 수정되었고 두 개의 시프트 레지스터를 활용해 데이터를 전달하며, 10의 자리, 1의 자리, 소수점 첫째 자리를 각각 개별적으로 처리한다. 또한, 시리얼 모니터에 온도를 출력하는 기능도 포함되어 있어 실시간으로 온도를 확인하거나 디버깅이 가능하다.

이 코드의 목적은 온도 데이터를 사용자에게 시각적(7-segment 디스플레이) 및 청각적(부저 경고음)으로 제공하여 시스템의 편의성과 실용성을 높이는 데 있다.

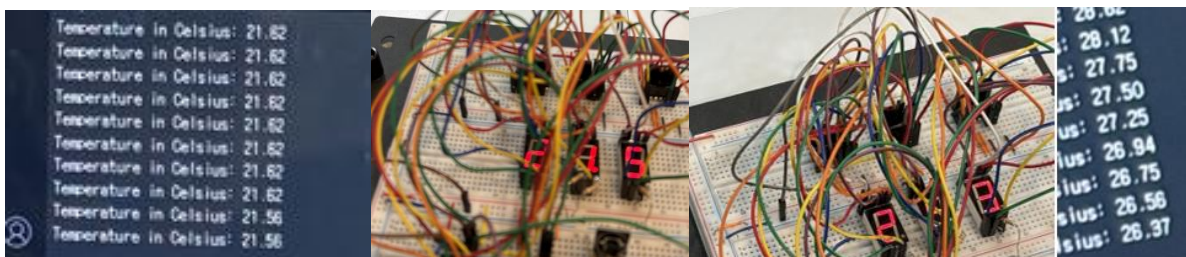
해결 후 시연 결과 사진



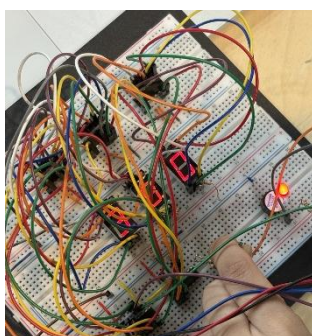
시연 결과

시리얼 모니터에서 출력된 온도와 일치하게 세그먼트에 값을 나타내며 온도가 30°C 이상일 경우 부저와 led가 깜빡이게 된다.

4. 최종 결과물



[그림1 시리얼 모니터에서 일치하게 세그먼트에 온도값을 출력한 사진]



[그림2 온도가 30°C 이상일 경우 부저와 LED가 작동하는 사진]

이 시스템은 DS18B20 온도 센서를 이용해 측정된 온도를 소수점 첫째 자리까지 정확하게 표시하고, 온도가 30°C 이상일 경우 부저가 0.5초 간격으로 3회 울리며 LED가 깜빡이는 경고 기능을 수행하도록 설계되었다. 온도 값은 정수부와 소수점 자리를 분리하여 각각 다른 FND에 표시하며, 두 개의 시프트 레지스터를 활용해 데이터를 처리한다. 또한, 실시간으로 온도를 확인할 수 있도록 시리얼 모니터를 통해 온도를 출력하며, 디버깅 및 모니터링이 용이하도록 구성되었다. 이 시스템은 설계한 기능을 모두 만족하였으며, 7-segment 디스플레이를 통한 정확한 온도 표시와 실시간 온도 모니터링을 성공적으로 구현하였다.

5. 고찰

-시행착오1 및 해결방안

변경결과: 이 과정에서는 회로와 코드의 간소화가 이루어졌다. 또한, 온도센서의 경우 아날로그 센서에서 디지털 센서로 변경하여 잡음에 강해지고, ADC 과정 없이 데이터 이동 횟수를 줄일 수 있었다. 디지털 센서로의 변경은 데이터 전송의 효율성을 높였으며, 이를 통해 시스템의 안정성이 향상되었다.

고찰

(1) IC 칩을 사용할 때, 사용되지 않는 핀이라 하더라도 데이터 시트를 참고하여 적절한 값을 입력해야 정상적으로 작동하는 점을 경험하였고 이를 통해 데이터 시트의 중요성을 다시 한 번 느낄 수 있었다.

(2) 브레드 보드를 사용하면서 접촉 불안정으로 인해 FND 출력 중 떨림 현상이 발생할 수 있다는 점을 인식하였고 이를 통해 실험 과정에서 연결 상태를 더 신경 써야 한다는 점을 깨닫게 되었다.

(3) 디지털 센서는 OneWire 통신 방식을 사용하여 VCC와 데이터 핀이 따로 분리되어 사용되지 않고, 하나의 핀에서 데이터를 전송한다. 하지만 일반적으로 디지털 센서는 날개로 판매되기보다는 모듈 형태로 제공되는데 이 모듈은 자체적으로 풀업 저항 등이 내장되어 있어 사용자는 별도로 저항을 추가할 필요 없이 간편하게 사용할 수 있다. 이로 인해 날개 센서에서 느낄 수 있었던 저항 추가 등의 불편함을 경험하지 않았지만, 직접적인 편리성을 실험을 통해 확인하는 기회는 없었다는 아쉬움이 있다.

(4) Arduino에서 출력되는 VCC를 여러 IC에 공급해야 하는 점에서 전원 불안정이 우려되었으나, 예상보다 안정적으로 작동하여 걱정했던 부분이 해결되었고 이로 인해 아두이노를 활용한 시스템 설계가 꽤 효율적으로 이루어졌음을 알 수 있었다.

-시행착오2 및 해결방안

변경결과: 시스템에 화재경보 기능이 추가되었으며, 온도는 소수점 첫째 자리까지 정확하게 표시되도록 개선되었다. 화재경보 기능은 온도가 특정 기준(예: 30°C)을 초과할 경우 부저와 LED를 통해 경고음을 발생시키고 시각적 신호를 제공하고 기존의 온도 표시 기능은 소수점 첫째 자리까지 확장되어, 더 정확한 온도 측정값을 7-segment 디스플레이에 출력할 수 있도록 하였다. 이를 통해 시스템은 화재 발생 시 경고 신호를 제공하며, 실시간으로 정확한 온도 모니터링이 가능해졌다.

고찰

(1) 온도를 정수 부분만 표시하였을 때는 온도가 올라가거나 내려가는 변화가 직관적으로 잘 보이지 않아 불편함이 있었으나, 소수점 첫째 자리까지 표시하도록 개선하면서 온도의 변화가 보다 명확하게 드러나 정확한 온도 측정이 가능해졌다. 즉, 소수점까지 표시함으로써 정확성과 실용성이 동시에 향상되었다는 것을 느꼈다.

(2) 화재경보 기능을 추가함으로써 기존의 단순 온도계를 넘어서 실용적인 기능을 갖추게 되었고,

온도 초과 시 부저와 LED를 통해 실시간 경고를 제공하는 등 시스템의 실용성이 크게 향상되었다.

(3) FND를 하나 더 추가하면, 온도 표시에서 소수점 위에 점을 올리고 섭씨 기호를 표시하는 기능을 추가할 수 있다는 아이디어도 떠올랐다.

(4) 다양한 개선을 통해 전체적인 작품의 완성도가 높아졌으며, 사용자가 보다 편리하고 직관적으로 온도를 모니터링하고 경고 기능을 활용할 수 있게 되었다.

6. 역할분담 및 프로젝트 진행과정

- 프로젝트 진행과정

연구내용	추진일정 (주)										
	5	6	7	8	9	10	11	12	13	14	15
주제선정 및 제안서 작성											
자료조사											
작품설계											
작품 수정											
최종보고서 및 프레젠테이션 작성											

- 역할 분담

박찬희	강서희	김가현	윤혜린
제안서 작성	중간보고서 작성	제안서 발표	제안서 PPT 제작
최종보고서 PPT 제작	초기 회로 구현	최종보고서 작성	회로도 및 아두이노 코드 수정
아두이노 초기 코드	부품 자료 조사	피스파이스 시뮬레이션 및 프로젝트 보조	부품 구매

프로젝트의 경우 대부분의 과정에서 조원들이 모두 함께 참여하였다.